



UNIVERSITY OF PISA
MASTER'S DEGREE IN CYBERSECURITY

STREAM CIPHER V6
AES GALOIS MULTIPLIER 3
COURSE HARDWARE AND EMBEDDED SECURITY

Author
Elena Palandri

Academic year 2025/2026

Contents

1	Project Specifications	1
1.1	The encryption	1
1.2	Design specifications	2
2	High-level Model	3
2.1	AES Galois Multiplier function	3
2.1.1	xtime function	3
2.1.2	Galois Multiplication-S	3
2.2	StreamCipherV6	4
2.2.1	Key initialization	5
2.2.2	Process - Encryption/Decryption	5
2.3	Test	5
3	RTL Design	8
3.1	Combinational Logic	9
3.2	Sequential Logic	9
4	Interface Specifications and Expected Behavior	11
4.1	Hardware Interface	11
4.1.1	Input Ports	11
4.1.2	Output Ports	12
4.2	Timing Protocol and Handshaking	12
4.3	Expected Behavior	12
4.3.1	Test Case 1: Simple 64-byte Message	13
4.3.2	Test Case 2: Empty Message	13
4.3.3	Test Case 3: Stalling - Interrupted Data Stream	13
5	Functional Verification	14
5.1	Waveform Analysis and Test Cases	14
5.1.1	Test Case 1: Simple 64-byte Message	14
5.1.2	Test Case 2: Empty Message	17

5.1.3 Test Case 3: Stalling - Interrupted Data Stream	17
6 FPGA Implementation Results	19
6.1 Results and Performance Metrics	19

List of Figures

1.1	Encryption scheme of the stream cipher.	2
2.1	Function S (Galois Multiplication) and xtime	4
2.2	Stream Cipher Class, including initialisation and processing (encryption/decryption)	6
2.3	Test Vectors	7
3.1	RTL architecture of the <code>stream_cipher_aes</code> module	8
4.1	Full waveform showing the ports and cases's results	12
5.1	Test Case 1: Simple 64-byte Message	15
5.2	Initial state and activation	16
5.3	Arrival of the <code>new_msg</code>	16
5.4	Valid input arrived	17
5.5	End of transmission	17
5.6	No Message provided in input	18
5.7	Interrupted data stream (<code>in_valid</code> goes low)	18

List of Tables

6.1 Hardware Implementation Results on Cyclone V 5CGXFC9D6F27C7 .	20
---	----

CHAPTER 1

Project Specifications

This project aims to design and implement a stream cipher based on a Galois Multiplication function derived from the Advanced Encryption Standard (AES) algorithm. The cipher must support both encryption and decryption processes.

1.1 The encryption

The encryption algorithm is defined by the following law: $C[i] = P[i] \oplus S(CB[i])$, where:

- $\oplus$ is the bitwise XOR operator
- $C[i]$ is the i^{th} byte of the ciphertext
- $P[i]$ is the i^{th} byte of the plaintext
- $CB[i]$ is the 32-bit value of the i^{th} counter block for $i = 0, 1, 2, \dots$ and it is formally defined by the equation $CB[i] = (K + i) \bmod 2^{32}$

K is the 32-bit symmetric key

$\bmod$ is the modulo operation

- $S()$ is the AES Galois Multiplication function of the AES algorithm. It accepts a 32-bit vector A as input and produces an 8-bit (1-byte) vector $f = S(CB[i]) = \text{xtime}(A[2] \oplus A[3]) \oplus A[3] \oplus A[0] \oplus A[1]$

$A[0]$, $A[1]$, $A[2]$, and $A[3]$ individual bytes of A ordered from most significant to least significant

$\text{xtime}()$ takes as input byte d (i.e. $d = A[0] \oplus A[1]$) and outputs a byte $e = \{d[6], d[5], d[4], d[3] \oplus d[7], d[2] \oplus d[7], d[1], d[0] \oplus d[7], d[7]\}$

{ } concatenation operator.

$d[7 : 0]$ represents the input byte composed of the 8 bits $d[j]$ for $j = 7, 6, 5, \dots, 1, 0$ with most significant bit ($d[7]$) and least significant bit ($d[0]$)

The encryption scheme 1.1 can be represented as follows:

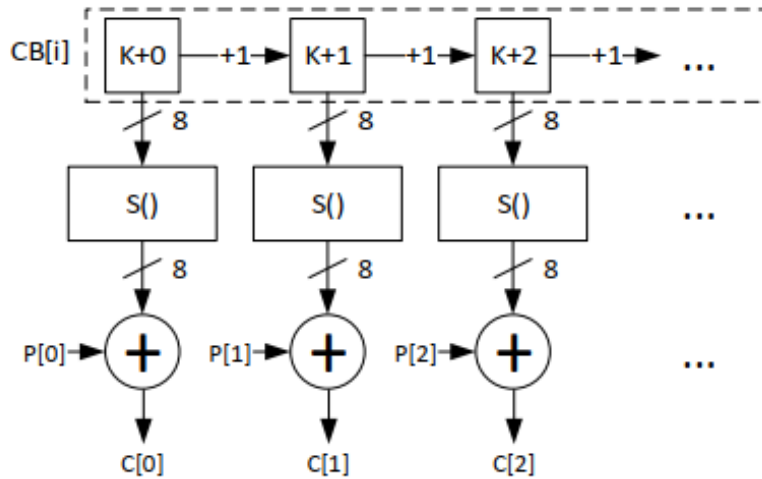


Figure 1.1: Encryption scheme of the stream cipher.

1.2 Design specifications

The project follows also these additional specifications:

- The module must include an asynchronous active-low reset port.
- The module interface must include a valid flag. The flag is driven to 1'b1 when the input data byte on the corresponding input port is valid, stable, and 1'b0 otherwise.
- The module interface shall include an output flag. The flag is driven to 1'b1 when the output data byte on the corresponding output port is ready and stable, 1'b0, otherwise.
- For each arbitrary plaintext and/or ciphertext message, the counter block must reset to the initial initialization key, K . Consequently, the interface requires an input flag to signal the beginning of a new message transmission.

In addition, it is suggested to follow these hints:

- For decryption, focus on the property of the XOR: $S \oplus S = 0$. Therefore, if the ciphertext is $C = P \oplus S$, the plaintext can be perfectly recovered by XORing the ciphertext with the same key stream: $C \oplus S = (P \oplus S) \oplus S = P \oplus (S \oplus S) = P \oplus 0 = P$
- For the implementation of the 32-bit counter and the associated 2^{32} modulo, the $(K + i) \bmod 2^{32}$, consider what happens to a 32-bit counter when the overflow occurs

CHAPTER 2

High-level Model

The High-level Model was created with Python.

2.1 AES Galois Multiplier function

The AES Galois Multiplication function of the AES algorithm accepts a 32-bit vector as input and produces an 8-bit vector $f = S(CB[i])$. The code is represented in the figure 2.1.

2.1.1 xtime function

The function `xtime` takes as input a byte d , which represents $d[7 : 0]$, the input byte composed of the 8 bits $d[j]$ for $j = 7, 6, 5, \dots, 1, 0$ with most significant bit $d[7]$ and $d[0]$ least significant bit. To extract the individual bits of the input vector d , it is necessary to shift right to push the desired byte to the rightmost position. Then, a bitwise $\&$ (AND) 1 is performed to assign to each bit its specific value (e.g., if d_0 has value 0, $0 \& 1$ will return 0, which will be put in d_0). This isolates the specific bit (e.g., d_0). After this, e is calculated as $e = \{d[6], d[5], d[4], d[3] \oplus d[7], d[2] \oplus d[7], d[1], d[0] \oplus d[7], d[7]\}$, associating to each bit that composes e , the correspondent value (e.g. $e_7=d_6$). Finally, all bits are shifted in their corresponding position, and then an $|$ (OR) is performed to combine all bits of e into a single byte and return e .

2.1.2 Galois Multiplication-S

The function S in the figure takes as input a 32-bit vector A . In particular, A is divided into 4 bytes, from the most significant byte, $A[0]$, to the least significant byte, $A[3]$. To extract these individual bytes from the input vector A , it is necessary to shift right to

push the desired byte to the rightmost position. Then, a bitwise $\&0xFF$ (with $0xFF$ representing 11111111) is performed to assign to each byte its specific value (e.g., if $a0$ has value 0, $0 \& 0xFF$ will return 0, which will be put in $a0$). This isolates the specific 8-bit byte (e.g., $A[0]$). In the end, the `xtime` function and the extracted bytes $A[0]$, $A[1]$, $A[2]$, and $A[3]$ are combined using the XOR operator to calculate $f = S(CB[i]) = xtime(A[2] \oplus A[3]) \oplus A[3] \oplus A[0] \oplus A[1]$. Then, f is returned and used as the keystream byte for the cipher.

```

1  # Stream Cipher v6 - AES Galois Multiplier 3
2
3  def xtime(d: int) -> int:
4
5      # Extract bits from the input byte 'd'
6      d0 = (d >> 0) & 1
7      d1 = (d >> 1) & 1
8      d2 = (d >> 2) & 1
9      d3 = (d >> 3) & 1
10     d4 = (d >> 4) & 1
11     d5 = (d >> 5) & 1
12     d6 = (d >> 6) & 1
13     d7 = (d >> 7) & 1
14
15     # e = {d[6], d[5], d[4], d[3]⊕d[7], d[2]⊕d[7], d[1], d[0]⊕d[7], d[7]}
16     e7 = d6
17     e6 = d5
18     e5 = d4
19     e4 = d3 ^ d7
20     e3 = d2 ^ d7
21     e2 = d1
22     e1 = d0 ^ d7
23     e0 = d7
24
25     # Combine the bits into a single byte
26     e = (e7 << 7) | (e6 << 6) | (e5 << 5) | (e4 << 4) | (e3 << 3) | (e2 << 2) | (e1 << 1) | (e0 << 0)
27
28     return e
29
30 # Galois multiplication
31 def S(a: int) -> int:
32
33     # Extract bytes from most significant A[0] to least significant A[3]
34     a0 = (a >> 24) & 0xFF
35     a1 = (a >> 16) & 0xFF
36     a2 = (a >> 8) & 0xFF
37     a3 = a & 0xFF
38
39     # Calculate f = xtime(A[2] ⊕ A[3]) ⊕ A[3] ⊕ A[0] ⊕ A[1]
40     f = xtime(a2 ^ a3) ^ a3 ^ a0 ^ a1
41     return f

```

Figure 2.1: Function S (Galois Multiplication) and `xtime`

2.2 StreamCipherV6

The class `StreamCipherV6` includes the initialization of the key, the processing (encryption/decryption), and exploits the AES Galois multiplication function as described in the subsections. The code is reported in figure 2.2.

2.2.1 Key initialization

The function `init` initializes the object's 32-bit key `K`. When the cipher is created, the function receives an integer value provided by the user to use as the key. As suggested, to make sure there is no overflow problem in the hardware, and the key has a fixed length of 32-bit (0xFFFFFFFF), it is necessary to perform a bitwise AND between the key and the 32-bit value. This operation is saved as `self.key` so the rest of the program can access it easily whenever it needs to encrypt or decrypt data.

2.2.2 Process - Encryption/Decryption

The function `process` represents both the encryption and decryption process by exploiting the involution property of the XOR operator. The function takes as input `data`, which will represent the ciphertext or the plaintext. The function must return bytes; this specification is necessary to ensure both input and output are the correct type, and the XOR operation is correctly taking and returning the right data type. Then, the `output` byte array is created to contain the output bytes. After this, a for-loop processes the single bytes in `data` and produces all i -th elements of the Counter Block, by:

- calculating the 32-bit Counter Block, defined by the equation $CB[i] = (K + i) \bmod 2^{32}$. To ensure the `CB_i` is exactly 32 bits, it is necessary to perform an AND with 0xFFFFFFFF, just like for the key initialization. The modulo $\bmod 32$ and bitwise AND with 0xFFFFFFFF ($2^{32} - 1$) are equivalent operations, however, the AND is much more computationally efficient.
- Then, the keystream is calculated using the function S , Galois multiplier, on CB_i as follows $S(CB[i])$.
- Given the advice on the property of the XOR ($S \oplus S = 0$), if the ciphertext is $C = P \oplus S$, the plaintext can be perfectly recovered by XORing the ciphertext with the same key stream: $C \oplus S = (P \oplus S) \oplus S = P \oplus (S \oplus S) = P \oplus 0 = P$. Therefore, it is possible to perform an XOR between the single byte of `data` and the keystream byte to obtain an output byte.
- At the end of the for, output is created by appending the output byte of each i 's to the array to obtain the whole output.

Finally, the bytes of the output are returned.

2.3 Test

To make sure the high-level code is correct, the results were tested by comparing the plaintext and the decrypted text. The figure 2.3 shows the code.

Then, Test Vectors were created to automate the verification process in ModelSim. A random key of exactly 4 bytes (32 bits) is created, then the bytes, ordered from most significant to least (Big Endian), are transformed into an integer and inserted in `K`. `K` is used to create the cipher thanks to the `StreamCipherV6`. After this, a random text of length 64 bytes is created and then ciphered, forming the ciphertext. Then, the desired directory "modelsim/tv/" is created if it is not already present.

```

43 class StreamCipherV6():
44
45     # Initialize the cipher with a fixed-length 32-bit key
46     def __init__(self, key: int):
47         self.key = key & 0xFFFFFFFF
48
49     # Process the input data byte by byte, applying the encryption/decryption law
50     def process(self, data: bytes) -> bytes:
51         output = bytearray()
52
53         for i, byte in enumerate(data):
54             # 32-bit Counter Block CB[i] = (K + i) mod 2^32
55             CB_i = (self.key + i) & 0xFFFFFFFF
56
57             # Generate f = S(CB[i])
58             f = S(CB_i)
59
60             # Encryption/decryption law: Data[i] ⊕ S(CB[i])
61             output_byte = byte ^ f
62             output.append(output_byte)
63
64         return bytes(output)

```

Figure 2.2: *Stream Cipher Class, including initialisation and processing (encryption/decryption)*

Then the code saves the generated key, plaintext, and ciphertext in separate text files in hexadecimal format, which can be used as test vectors for ModelSim. The files are all saved in the "modelsim/tv/" folder. Using the "write" mode "w" makes sure that if the file already exists, it will be overwritten; if the file does not exist, it will be created. In ModelSim, \$readmemh() requires the file to have a single value for each line, with the right dimension required by SystemVerilog. In particular, the key is saved as an 8-digit hexadecimal number $K:08X$ (for 32-bit). The plaintext and ciphertext are saved as 2-digit hexadecimal numbers for each byte $byte:02X$ (for 8-bit). For both texts, the code iterates through each byte in the respective text and writes it to the file in hexadecimal format, ensuring that each byte is represented as a two-digit hexadecimal number. These formats allow for easy comparison, verification, and testing in ModelSim.

```

78 # Test Vectors for ModelSim
79 import os
80
81 if __name__ == "__main__":
82
83     # Generates a random key 32-bit (4 byte)
84     random_key_bytes = os.urandom(4)
85     K = int.from_bytes(random_key_bytes, byteorder='big')
86
87     cipher = StreamCipherV6(K)
88
89     # Generates a random plaintext of random lenght (under 64 bytes)
90     num_test_bytes = 64
91     plaintext = os.urandom(num_test_bytes)
92
93     ciphertext = cipher.process(plaintext)
94
95     current_script_dir = os.path.dirname(os.path.abspath(__file__))
96     target_dir = os.path.abspath(os.path.join(current_script_dir, "..", "..", "project_930II_2025_2026_palandri", "modelsim", "tv"))
97     os.makedirs(target_dir, exist_ok=True)
98
99     # {K:02X} HEX conversion of 2 cipher bytes
100    # {K:08X} creates the 8-digit hexadecimal representation of the key, zero-padded if necessary
101
102    # Key saving
103    with open(os.path.join(target_dir, "key.txt"), "w") as f_key:
104        f_key.write(f"{K:08X}\n")
105
106    # Plaintext saving
107    with open(os.path.join(target_dir, "ptxt.txt"), "w") as f_ptxt:
108        for byte in plaintext:
109            f_ptxt.write(f"{byte:02X}\n")
110
111    # Cyphertext saving
112    with open(os.path.join(target_dir, "ctxt.txt"), "w") as f_ctxt:
113        for byte in ciphertext:
114            f_ctxt.write(f"{byte:02X}\n")

```

Figure 2.3: *Test Vectors*

CHAPTER 3

RTL Design

The RTL (Register Transfer Level) architecture of the hardware module `stream_cipher_aes` was designed in SystemVerilog and represented graphically in Microsoft Visio. The figure 3.1 illustrates the design.

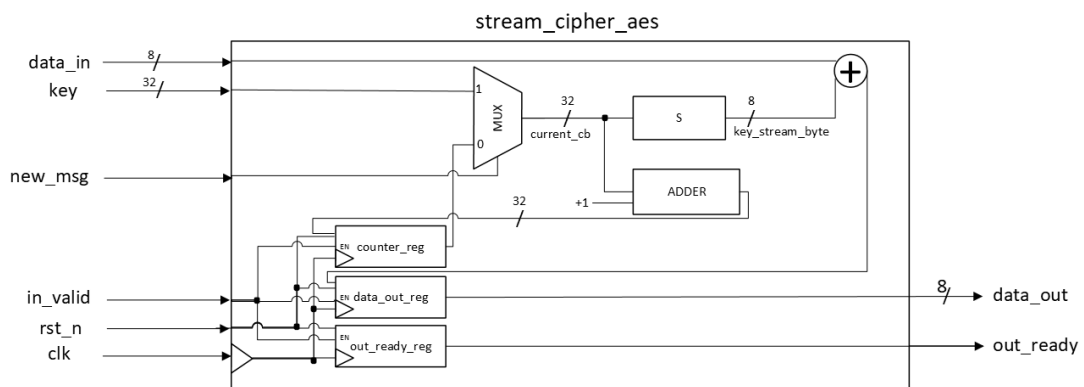


Figure 3.1: RTL architecture of the `stream_cipher_aes` module

Here, the typical sequential execution of the high-level software model is replaced with the concurrency of hardware. This allows the creation of a much more efficient architecture, where the module can complete complex operations as the Galois Multiplication in a single clock cycle.

In the figure 3.1, the following words and symbols are used:

- MUX: Multiplexer, which selects between two input signals and forwards the chosen signal to a single output

- S: Represents the Galois Multiplication function
- ADDER: An arithmetic logic unit performing a +1 increment
- EN: Enable signal
- $\oplus$: XOR or Exclusive OR, the logic operator used to encrypt and decrypt a text
- $\triangleright$: Represents the clock signal of an edge-triggered sequential logic block
- •: Represents the split a line into multiple lines
- /: Represents the length of the data going through the signal path

The architecture is contained in a block called `stream_cipher_aes` module. The left boundary features the input ports: `data_in`, `key`, `new_msg`, `in_valid`, `rst_n`, and `clk`. The right boundary features the output ports: `data_out` and `out_ready`. Both input and output specifications are thoroughly described in Chapter 4.

3.1 Combinational Logic

The combinatorial logic elaborates the signal and executes the cryptographic calculations. In the diagram, there are:

- MUX is controlled by the selection pin `new_msg`. It selects `current_cb`. If the `new_msg` is high (1), the MUX returns the key value; if `new_msg` is low (0), it returns the value calculated by the internal counter register
- S represents the mathematical cryptographic function. It receives as input the `current_cb` of 32-bit length and executes calculations. In the end, it produces as output the 8-bit `key_stream_byte`. It is represented as a simple white box for simplicity, however it contains all the buses and XOR logic gates that represent the expressions and the `xtime` function (which is also composed by buses and XOR logic gates) defined in Chapter 1.
- ADDER increments the value of `current_cb` by one unit (performs a +1), preparing it to update the `counter_reg` (counter register) at the next clock cycle.
- XOR makes a bitwise XOR-operation between the 8-bit `data_in` and the `key_stream_byte`, which has the same length. Its output is put in the `data_out_reg`

Separating the combinational and sequential logic aims to optimise the synthesis process. Quartus can map the multiplexer, adder, Galois Multiplier and the XOR operations in a much more efficient way. This strict separation also prevents unintentional latches and optimises delays in the critical path, maximising the frequency.

3.2 Sequential Logic

The registers contain edge-triggered logic, indicated by the clock triangle on the bottom left of the blocks.

There are three registers:

- `counter_reg`: It is a 32-bit register that saves the updated value of the CB calculated by the ADDER
- `data_out_reg`: It is an 8-bit register that gets the XOR output at each clock cycle and returns the `data_out` as the output of the `stream_cipher_aes` module
- `out_ready_reg`: It is a 1-bit register that saves the `in_valid` control signal to synchronize the `out_ready` flag and the output data, confirming its validity to the external modules

The update of these registers is synchronized by the rising edge of the `clk` signal and initialised by an active low asynchronous reset, `rst_n`. The sequential logic manages the synchronisation of the whole module. The flag `in_valid` controls the three registers (`counter_reg`, `data_out_reg`, `out_ready_reg`), by updating their state only when `in_valid` is high. This simultaneous update permits to save resources during inactivity periods and guaranteeing the integrity of data during the transit and allows to implement a handshake mechanism.

CHAPTER 4

Interface Specifications and Expected Behavior

This chapter acts as a functional user manual for the `stream_cipher_aes` module. It defines the hardware interface and timing protocol necessary to interact correctly with the module, and the expected behaviour of the system in three use cases.

4.1 Hardware Interface

The module interacts with external systems through a strictly synchronous interface, except for the asynchronous reset `rst_n`. The input and output ports are defined as follows:

4.1.1 Input Ports

- `clk` (1-bit): The system clock, which coordinates all internal state transitions and data sampling happening on its rising edge.
- `rst_n` (1-bit): An active-low asynchronous reset, which, when driven low (has value 0), instantly initializes the internal counter and output registers, regardless of the clock state. After the initialization, it must be kept high (has value 1).
- `in_valid` (1-bit): The input data validation flag, which is driven high to indicate that the data `data_in` is stable and ready to be processed.
- `new_msg` (1-bit): The initialization flag, which is driven high together with the first byte of a new message to make the multiplexer load the 32-bit `key` into the CB.
- `key` (32-bit): The symmetric cryptographic key, which must be stable when `new_msg` is high.

- `data_in` (8-bit): The input bus for the plaintext or ciphertext stream, which is sampled only if `in_valid` is high.

4.1.2 Output Ports

- `out_ready` (1-bit): The output data validation flag, which is driven high to signal that the computation has been completed and the byte on `data_out` is valid and can be read by external systems.
- `data_out` (8-bit): The output bus containing the processed bytes of the ciphertext or plaintext.

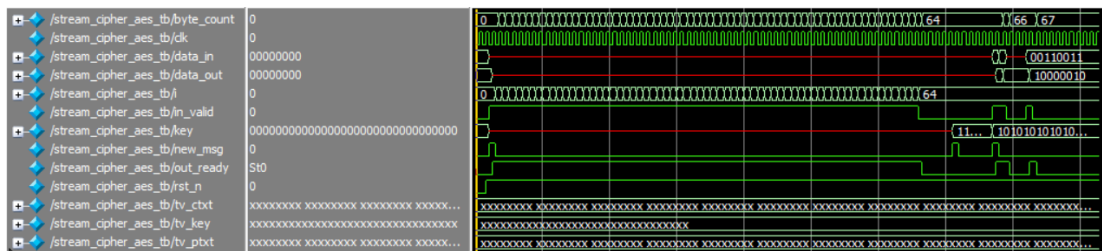


Figure 4.1: Full waveform showing the ports and cases's results

4.2 Timing Protocol and Handshaking

To guarantee the integrity of the processing of the data, external modules follow this protocol:

1. **System Initialization:** Upon power-up, `rst_n` must be set low and then set high to initialize the hardware.
2. **Starting a New Message:** To process a new message, the external system must place the 32-bit symmetric key in `key`, place the first byte of the message on `data_in`, and set both `in_valid` and `new_msg` high.
3. **Continuous Streaming:** At the next clock cycle, `new_msg` must be pulled low to allow the internal counter to increment. The module takes one byte per clock cycle as long as `in_valid` remains high.
4. **Data Retrieval:** Due to the sequential registers, the module has a one-clock-cycle latency; in fact, it will assert `out_ready` high provide the processed byte to `data_out` exactly one clock cycle after sampling a valid `data_in`.

4.3 Expected Behavior

The following sections illustrate the module's response to various input conditions, covering both standard operations and corner cases.

4.3.1 Test Case 1: Simple 64-byte Message

This test case, shown in Figure 5.1, demonstrates the standard transmission of a continuous message.

The transmission starts when the user provides the key and asserts `new_msg`. The module responds exactly one clock cycle later by raising `out_ready` and giving the first valid byte as output. The system maintains a throughput of one byte per clock cycle as long as `in_valid` is high. When the transmission finishes, and the user drops low `in_valid`, the module pulls `out_ready` low on the next clock cycle, concluding the handshake.

4.3.2 Test Case 2: Empty Message

This test case, shown in Figure 5.6, demonstrates the hardware behavior when a new key is loaded but no data is provided.

The user asserts `new_msg` to load a new key but keeps `in_valid` low. The module updates the internal key register via the multiplexer but does not process any data. As expected, `out_ready` correctly remains low, preventing the generation of invalid or false outputs.

4.3.3 Test Case 3: Stalling - Interrupted Data Stream

In real-world applications, a transmitting system might not be persistent and have data ready at every consecutive clock cycle. This test case, illustrated in Figure 5.7, demonstrates the robustness of the handshaking protocol when the data stream is interrupted.

The transmission begins as in the first case, but `in_valid` drops to 0 during the message's transmission. The hardware instantly disables the internal registers and pauses the counter's incrementation. Then, `out_ready` drops low in the following clock cycle, preventing the external system from reading duplicated or invalid data. The module holds its state and resumes processing only when the user raises `in_valid` high again.

CHAPTER 5

Functional Verification

This chapter describes the testing of the module, the testbench architecture, and what tests are performed to verify the functionality of the module, according to the expected behavior described in Chapter 4.

The functional verification proves the algorithmic correctness of the ciphertext and tests the behaviour of the module in both valid streams and boundary conditions (the two corner cases) without performing hard system resets. This proves the recovery capabilities of the architecture and the robustness of the handshaking protocol.

5.1 Waveform Analysis and Test Cases

ModelSim's testbench was created to test the expected behavior of three scenarios:

- Simple 64-byte Message
- Empty Message
- Stalling

The figure 4.1 from Chapter 4 illustrates the module's response to various input conditions.

5.1.1 Test Case 1: Simple 64-byte Message

This test case aims to show the "perfect" case, in which a random message of 64 bytes (created with Python's Test Vectors) is sent. The initialization occurs when `new_msg` is created. Then, a stream of bytes is present as long as `in_valid` is high. The processed bytes are output on the following rising clock edge, maintaining an efficient

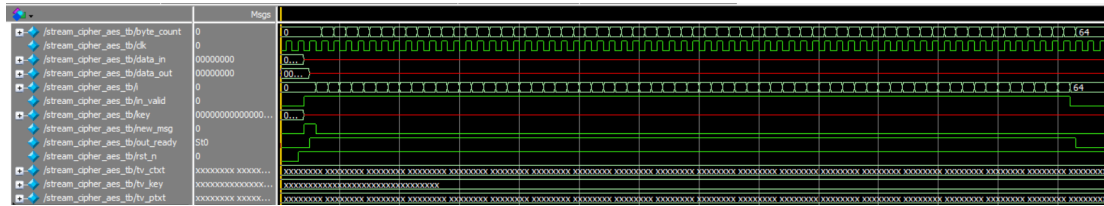


Figure 5.1: Test Case 1: Simple 64-byte Message

throughput (one byte per cycle) without losing any cycles. From the waveform, it is possible to see:

- The `byte_count` is updated only once the `new_message` value returned to low, implementing the handshake mechanism
- The `clk` signal maintains a period of 10 ps
- The `data_in` is produced right from the beginning, and provides valid 8-bit data during the active transmission phase. After this, it transitions to an undefined state.
- The `data_out` is also produced from the beginning, it is 8-bit long and has a one-cycle latency, in fact, it terminates at the following rising edge with respect to `data_in`. After this, it transitions to an undefined state.
- The `i` begins at the 3rd falling edge of the clock and follows the same pattern as `byte_count`,
- The `in_valid` goes from low to high at the second falling edge of the clock and stays high till the `i` the 64-byte are fully processed
- The `key` is sent in during the first two clock cycles, then it transitions to an undefined state till 710 ps
- The `new_msg` goes from low to high at 20 ps, after the second clock cycle. It remains active for one clock cycle, loading the key, and then returns to low, at 30 ps. In the end, once `key` is also high, it stays high for another clock cycle
- The `out_ready` goes from low to high at 25 ps, confirming that the first output byte is available. Once the data out is terminated, on the rising edge of the clock. It stays high till 665 ps, once `byte_count` reaches the 64th byte
- The `rst_n` (asynchronous reset) goes from low to high after 15 ps, at the second rising edge of the clock, and stays high the whole time
- The hexadecimal value of the test vector `tv_ctxt` is reported
- The hexadecimal value of the test vector `tv_key` is reported
- The hexadecimal value of the test vector `tv_ptxt` is reported

In particular, here are reported four key moments of the Test Case's Waveform and the corresponding figure.

Test Case 1.1: Reset and Initialization

The vertical line in 5.2 highlights the initial instant of the program, where the signal `rst_n` is low. This condition zeroes all internal registers, control signals, and data buses. All test vectors (ciphertext, key, and plaintext) are correctly allocated in the memory. Everything is ready for execution.

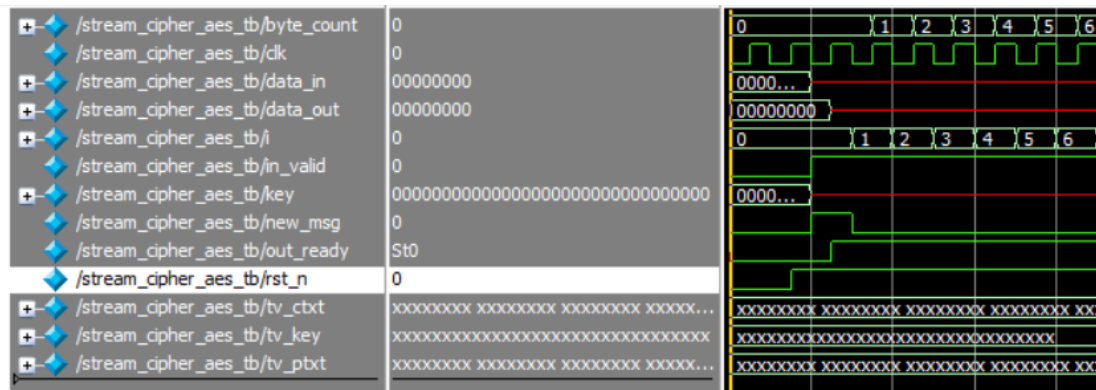


Figure 5.2: *Initial state and activation*

Test Case 1.2: Start of Transmission

The vertical line in the figure 5.3 highlights the instant of 20 ps, when `new_msg` and `in_valid` rise to high, on the falling edge of the clock signal. At the same time, the `key` finishes the reception of the symmetric key.

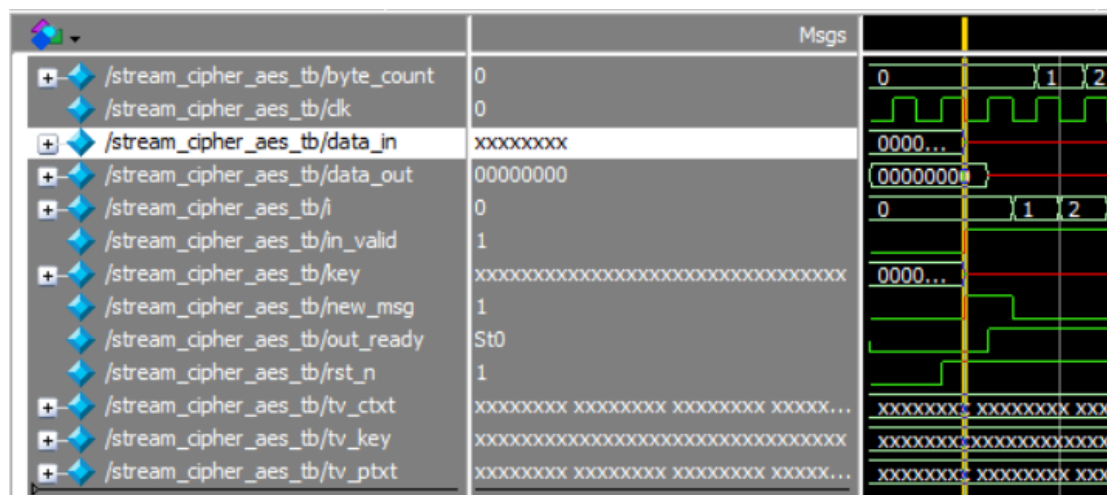


Figure 5.3: *Arrival of the new_msg*

Test Case 1.3: Valid input

The vertical line in the figure 5.4 highlights the instant at 25 ps. At the rising edge of the clock, `out_ready` rises high. `data_out` presents its first valid ciphred byte. At the following falling edge of the clock, `new_msg` goes back to 1. This deactivates the key insertion in the multiplexer, allowing the internal CB to start the incrementation.

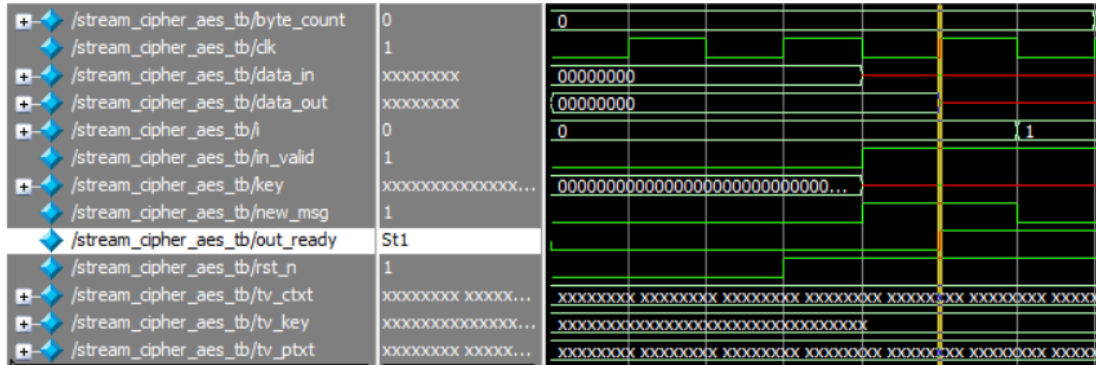


Figure 5.4: *Valid input arrived*

Test Case 1.4: End of transmission

The vertical line in the figure 5.5 highlights the instant at 665 ps, on the last rising edge of the test case. The `byte_count` verifier reached the target value of 64. The `in_valid` went low at the previous falling edge, as the input message ended. Therefore, `out_ready` falls to 0, terminating the handshaking process. No error is found.

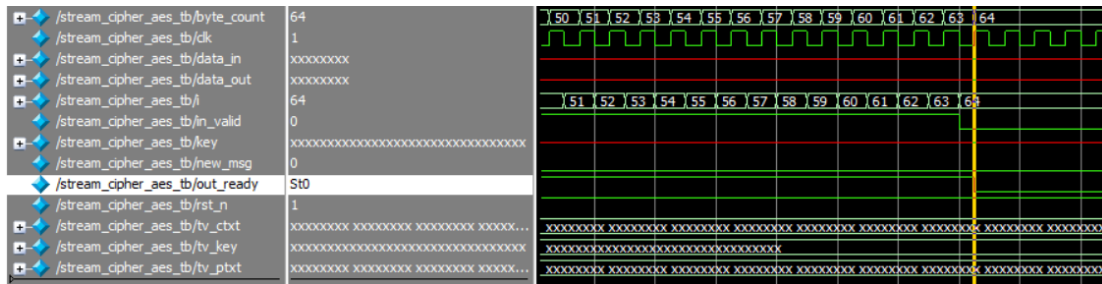


Figure 5.5: *End of transmission*

The waveform analysis of the simple and continuous stream confirms the effectiveness of the architecture. Following an initial one-cycle latency required by the registers, the system shows a perfectly synchronised pipeline, maintaining a steady throughput of one byte per clock cycle.

5.1.2 Test Case 2: Empty Message

The vertical line in the figure 5.5 highlights the beginning of the test, on the falling edge of the clock. The `new_msg` rises to high and `key` receives a new sequence (predetermined, just for testing). The `in_valid` remains voluntarily low, and the multiplexer updates the CB with a new key, but without validation, `out_ready` remains low. This makes sure that the system does not create wrong outputs.

5.1.3 Test Case 3: Stalling - Interrupted Data Stream

The vertical line in the figure 5.6 highlights the beginning of a new transmission. `new_msg` and `in_valid` both rise high. The system receives a new key and data bytes on `data_in`. After this, it is possible to notice that `in_valid` goes low. The

CHAPTER 6

FPGA Implementation Results

This chapter details the use of the program Intel Quartus Prime Lite Edition, the logic synthesis and physical implementation of the `stream_cipher_aes` modelsim's module. In particular, using the **5CGXFC9D6F27C7** FPGA device, part of the Cyclone V family, as per the project specifications.

The workspace was initialised using the Python scripts `build.py` and `build.py`. The last one was used to generate the Quartus project (`.qpf`), including the SystemVerilog design files and the timing constraints from the `constr/` directory. The Design was compiled and then started the: Analysis & Synthesis, Fitter (Place & Route), Assembler (Generate programming files) and finally, Timing Analysis.

6.1 Results and Performance Metrics

The final evaluation of the hardware implementation is based on: Area (A), Maximum Frequency (F_{max}), Cycles Per Data (CPD), Throughput (THR), and Efficiency (E).

To evaluate the maximum performance of the module, a Static Timing Analysis (STA) was performed using Timing Analyzer. The value of F_{max} was extracted by analyzing the PVT (Process, Voltage, Temperature) corners generated by the STA. The lowest frequency (147.54 MHz) was chosen from the Slow 110mV 0C Model as the official maximum frequency to guarantee the module's correct behaviour, even under worst-case scenarios.

The throughput is calculated using the formula $THR = \frac{Q}{CPD} \cdot F_{max}$, where $Q = 8$ bits (1 byte processed per operation) and $CPD = 1$, since the unrolled/pipeline architecture of the cipher outputs one byte per clock cycle after the initial latency. Finally, the hardware efficiency is computed as $E = \frac{THR}{A}$.

The results are summarized in Table 6.1.

Table 6.1: *Hardware Implementation Results on Cyclone V 5CGXFC9D6F27C7*

Metric	Value
Resources / Area (A)	64 ALMs
Maximum Frequency (F_{max})	147.54 MHz
Cycles Per Data (CPD)	1 cc
Data Width (Q)	8 bits
Throughput (THR)	1180.32 Mbps
Efficiency (E)	18.44 Mbps/ALM

The results show that the module achieves a good balance between resource utilization and processing speed. The 64 ALMs (Adaptive Logic Modules), the high throughput of over 1.1 Gbps achieved and the efficiency of 18.44 Mbps/ALM prove that the design is efficient and all the desired constraints are respected.